

组号：_____

本科实验报告

TurtleBot4 移动机器人自主导航与避障

Experiment 2: Autonomous Navigation of TurtleBot4

课程名称：_____ 机器人与智能系统综合实践

姓 名：_____ 请填写

院 系：_____ 请填写

专 业：_____ 请填写

学 号：_____ 请填写

指导老师：_____ 请填写

实验平台：_____ ROS 2 Humble / TurtleBot4

2026 年 7 月 11 日

摘 要

本实验面向 TurtleBot4 移动机器人，在 Ubuntu 22.04 与 ROS 2 Humble 环境下构建 Ignition Gazebo 仿真系统，完成增量式环境建图、基于静态地图的自定位、代价感知 A* 全局路径规划以及 Nav2 闭环自主导航。系统使用 TurtleBot4 的差速移动底盘模型、RPLIDAR 二维激光雷达、里程计与 TF 坐标变换；SLAM 阶段由 SLAM Toolbox 建立占据栅格地图并保存为 PGM/YAML，导航阶段由 AMCL 完成 map 坐标系下的位姿估计。

为满足全局路径规划算法的编程要求，本实验按照 `nav2_core::GlobalPlanner` 接口实现 C++ A* 插件。算法在八连通栅格上采用 octile 启发函数，禁止对角穿角，并将代价地图栅格值引入边权，从而优先选择具有安全间隙的可行路径。实测生成的全局路径包含 189 个位姿，长度为 6.838 m，最大横向绕行为 2.425 m，路径绕过 `shelf_7` 占用区域。相同 Nav2 目标 UUID 的动作状态由 EXECUTING (2) 转为 SUCCEEDED (4)，未出现 ABORTED (6)，机器人最终距目标 0.215 m。

SLAM 流程中机器人里程计位移为 0.550 m，已知栅格数由 41,067 增至 45,925，增量为 4,858 个栅格，地图分辨率为 0.05 m/cell。受测试主机仅有约 2 个 CPU 核心且使用 `llvmpipe` 软件渲染的限制，导航阶段曾出现一次超过 10 s 的仿真时钟监听间隙；时钟随后恢复并继续递增，导航动作最终成功。该现象被作为性能限制保留，不影响“在 Ignition Gazebo 中实现 TurtleBot4 自主导航”的功能性结论。

关键词：TurtleBot4；ROS 2；Ignition Gazebo；SLAM Toolbox；AMCL；Nav2；A*；自主导航

目录

摘要	I
插图目录	III
表格目录	IV
1 实验目的与要求	1
1.1 实验目的	1
1.2 仿真实验要求	1
1.3 实验范围说明	1
2 实验平台与 TurtleBot4 工作原理	2
2.1 软硬件与仿真平台	2
2.2 TurtleBot4 主要硬件构成	2
2.3 远程主机连接原理	3
3 自主导航系统设计	3
3.1 总体架构	3
3.2 SLAM Toolbox 增量建图	4
3.3 AMCL 自定位	4
3.4 代价感知 A* 全局路径规划	4
3.5 Nav2 闭环执行	5
4 软件实现	5
4.1 代码包结构	5
4.2 仿真启动模式	6
4.3 关键导航参数	6
5 实验步骤	6
5.1 编译与静态验证	6
5.2 SLAM 增量建图	7
5.3 AMCL 初始化与规划器检查	7
5.4 发送导航目标	7
6 实验结果与分析	8

6.1	SLAM 增量建图结果	8
6.2	AMCL 自定位结果	10
6.3	A* 路径与绕障结果	11
6.4	Nav2 导航终态	13
6.5	仿真时钟与低负载配置分析	14
7	问题诊断与改进	15
7.1	启动参数作用域	15
7.2	RViz 视角确定性	15
7.3	后续优化方向	15
8	结论	15
A	仿真复现命令	18
A.1	导航模式	18
A.2	建图模式	18
A.3	低负载复测模式	18
B	报告生成方法	18

插图

1	TurtleBot4 仿真自主导航系统架构	3
2	SLAM 建图运动前的局部地图	8
3	SLAM 建图运动后的增量地图	9
4	增量建图客观指标	9
5	SLAM 流程保存的占据栅格地图	10
6	通过 RViz 设定初始位姿后的 AMCL 收敛状态	11
7	A* 全局路径与 shelf_7 绕障判定	12
8	RViz 中的 A* 路径与导航执行状态	12
9	Nav2 到达目标: Feedback 为 reached, 剩余距离为 0.00 m	13

表格

1	实验平台配置	2
2	仿真代码包主要文件	5
3	自定义 A* 与 Nav2 关键参数	6
4	SLAM 流程量化结果	10
5	全局路径量化结果	13
6	仿真自主导航最终验收	14

1 实验目的与要求

1.1 实验目的

1. 了解 TurtleBot4 机器人的主要硬件构成及其工作原理；
2. 掌握 TurtleBot4 与远程主机之间的连接原理；
3. 掌握 Ignition Gazebo 与 RViz 仿真软件的使用；
4. 掌握 ROS 2 机器人操作系统的基本操作；
5. 掌握移动机器人自主导航功能的基本实现原理。

1.2 仿真实验要求

在 Ignition Gazebo 仿真环境中实现 TurtleBot4 机器人的自主导航。仿真内容包括：

1. 调用机器人控制系统内置的 SLAM 模块对周围环境进行地图建模；
2. 调用自定位模块，在地图坐标系下获得机器人位姿；
3. 编程实现全局路径规划算法，并接入导航系统；
4. 在 RViz 中设定初始位姿和导航目标，使机器人在仿真环境中自主运动并绕开障碍；
5. 采集地图、路径、动作状态和最终位姿等客观证据，分析实验结果。

1.3 实验范围说明

本报告仅覆盖课程要求中的**仿真部分**。真实 TurtleBot4、实验室无线网络和 Discovery Server 不在当前测试环境中，因此不将实机连接或实机自主导航列为本报告的完成项。代码包中保留了实机启动入口，便于后续将相同 A* 插件和 Nav2 参数迁移到实体机器人。

2 实验平台与 TurtleBot4 工作原理

2.1 软硬件与仿真平台

表 1 实验平台配置

项目	配置
操作系统	Ubuntu 22.04 LTS
机器人中间件	ROS 2 Humble Hawksbill
仿真器	Ignition Gazebo / Gazebo Sim, TurtleBot4 warehouse 世界
可视化工具	RViz2, 实验专用固定俯视配置 <code>experiment.rviz</code>
建图与定位	SLAM Toolbox; Nav2 Map Server; AMCL
导航框架	Nav2 BT Navigator、Planner Server、DWB Controller、Velocity Smoother
全局规划算法	自研 <code>tb4_astar_planner/AStarPlanner</code> pluginlib 插件
测试主机	约 2 vCPU、7.8 GiB 内存、无独立 GPU, OpenGL 使用 llvmpipe

2.2 TurtleBot4 主要硬件构成

TurtleBot4 标准版以 iRobot Create 3 为移动底盘，采用两轮差速驱动并通过轮速编码器与惯性测量信息生成里程计；上位计算单元为 Raspberry Pi 4B，负责运行 ROS 2 节点、传感器驱动与高层任务。标准传感器包含 RPLIDAR A1M8 360° 二维激光雷达和 OAK-D-Pro 深度视觉相机。RPLIDAR 输出的 `/scan` 是本实验建图、定位和局部避障的主要环境观测；Create 3 底盘输出 `/odom`，并接收 `/cmd_vel` 速度指令。

差速底盘的平面运动学可写为

$$v = \frac{r}{2} (\omega_R + \omega_L), \quad \omega = \frac{r}{L} (\omega_R - \omega_L), \quad (1)$$

其中 r 为驱动轮半径， L 为轮距， ω_R 与 ω_L 分别为左右轮角速度。机器人平面位姿由 x 、 y 和航向角 θ 构成，其连续时间模型为

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \omega. \quad (2)$$

仿真器对上述运动学、碰撞和传感器过程进行数值积分，ROS 2 节点通过相同接口使用仿真数据，因此算法层与实机层可以共享。

2.3 远程主机连接原理

实体 TurtleBot4 包含 Create 3 与 Raspberry Pi 两个计算单元，两者通过 USB-C 建立供电和以太网连接；远程 PC 通过 Wi-Fi 加入机器人网络。ROS 2 使用 DDS 进行节点发现与数据分发，TurtleBot4 可使用 Simple Discovery 的组播发现，也可由 Fast DDS Discovery Server 集中管理参与者。远程主机需要与机器人使用兼容的 ROS_DOMAIN_ID、RMW 实现和 Discovery Server 配置，随后即可订阅 `/scan`、`/odom`，并调用 Nav2 action。

本次单机仿真中，Gazebo、SLAM、AMCL、Nav2 和 RViz 位于同一 ROS 2 域内，节点通信机制与远程连接一致，只是数据链路由本机回环网络代替 Wi-Fi。由此可以在没有实体硬件时验证 ROS 2 接口、消息类型、TF 树和 action 生命周期。

3 自主导航系统设计

3.1 总体架构

系统按“仿真环境与传感器—建图/定位—全局规划—局部控制—底盘执行”的分层结构设计，如图 1 所示。项目启动文件将官方 TurtleBot4 Ignition 启动项放入隔离作用域，关闭官方启动文件内部的 SLAM、定位和 Nav2，随后由本项目按模式启动唯一一套建图或定位节点以及自定义 Nav2 参数，避免重复节点和 launch 参数相互覆盖。

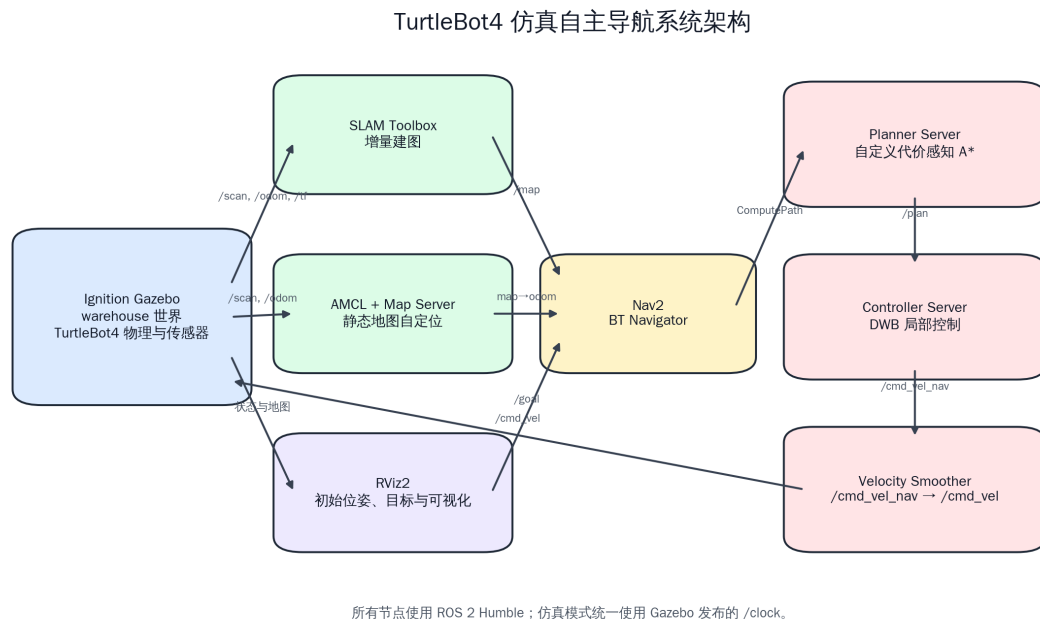


图 1 TurtleBot4 仿真自主导航系统架构

3.2 SLAM Toolbox 增量建图

SLAM 阶段由激光扫描与里程计估计机器人轨迹，并建立二维占据栅格地图。对于栅格 m_i ，占据概率可用 log-odds 形式递推：

$$l_{t,i} = l_{t-1,i} + \log \frac{p(m_i | z_t, \mathbf{x}_t)}{1 - p(m_i | z_t, \mathbf{x}_t)} - l_{0,i}, \quad (3)$$

其中 z_t 为第 t 帧激光观测， $\mathbf{x}_t$ 为机器人位姿。SLAM Toolbox 同时维护激光关键帧和位姿图，通过扫描匹配与图优化校正累积误差，并发布 map -> odom 变换。实验以同步模式运行，建图后使用 nav2_map_server/map_saver_cli 保存地图。

3.3 AMCL 自定位

导航阶段加载与 warehouse 世界对应的静态地图，AMCL 使用粒子滤波估计机器人在 map 坐标系中的位姿。粒子集合 $\{\mathbf{x}_t^{[k]}, w_t^{[k]}\}_{k=1}^N$ 依次经历运动采样、激光观测加权和重采样。在 RViz 中通过 2D Pose Estimate 发布 /initialpose，使粒子在初始位置附近收敛；AMCL 随后发布 map -> odom，与底盘的 odom -> base_link 共同构成导航所需 TF 链。

3.4 代价感知 A* 全局路径规划

自定义规划器继承 nav2_core::GlobalPlanner，由 pluginlib 导出并在 Planner Server 中注册为 GridBased。算法在全局代价地图上执行 A* 搜索，节点评价函数为

$$f(n) = g(n) + h(n), \quad (4)$$

其中 $g(n)$ 是从起点到节点 n 的累计代价， $h(n)$ 是到目标的启发式估计。八连通栅格采用 octile distance：

$$h(n) = \sqrt{2} \min(\Delta x, \Delta y) + \max(\Delta x, \Delta y) - \min(\Delta x, \Delta y). \quad (5)$$

为使路径远离膨胀代价较高的障碍边缘，进入栅格 n 的边权定义为

$$c(n) = d \left(1 + \lambda \frac{\min(C(n), 252)}{252} \right), \quad (6)$$

其中 d 为直移距离 1 或对角距离 $\sqrt{2}$ ， $C(n)$ 为 Nav2 代价地图值， $\lambda = 2.0$ 为代价惩罚系数。算法将代价不小于 253 的栅格视为致命障碍，并禁止未知区域。对角扩展时

同时检查相邻两个正交栅格，任一不可通行则拒绝该对角边，从而避免路径从障碍角点之间穿过。

3.5 Nav2 闭环执行

RViz 的 Nav2 Goal 工具向 BT Navigator 发送 `NavigateToPose` action。Planner Server 调用 A* 生成 `nav_msgs/Path`，Simple Smoother 对路径进行平滑；DWB 局部控制器结合局部代价地图产生 `/cmd_vel_nav`，Velocity Smoother 以 20 Hz 限制速度和加速度后输出 `/cmd_vel`。机器人运动产生新的激光与里程计，形成完整反馈闭环。

4 软件实现

4.1 代码包结构

表 2 仿真代码包主要文件

路径	功能
<code>tb4_astar_planner/src/astar_search.cpp</code>	与 ROS 解耦的八连通代价感知 A* 搜索核心。
<code>tb4_astar_planner/src/astar_planner.cpp</code>	Nav2 GlobalPlanner 适配、坐标转换、路径姿态生成和 pluginlib 导出。
<code>tb4_astar_planner/global_planner_plugin.xml</code>	插件类型与基类注册信息。
<code>tb4_astar_planner/test/test_astar_search.cpp</code>	空地图、墙体缺口、无路、禁止穿角和高代价绕行单元测试。
<code>tb4_experiment_bringup/launch/simulation.launch.py</code>	Ignition、SLAM/AMCL、Nav2 与 RViz 的统一仿真入口。
<code>tb4_experiment_bringup/config/nav2_astar.yaml</code>	Planner、Controller、Costmap、Smoother 与 BT Navigator 参数。
<code>tb4_experiment_bringup/config/experiment.rviz</code>	固定 map frame、俯视方向、初始位姿工具和目标工具。

路径	功能
maps/lab_map.yaml, maps/ lab_map.pgm	SLAM 流程保存的增量地图。

4.2 仿真启动模式

统一入口通过 `slam` 参数提供两个互斥流程：

- `slam:=true`：启动一套 SLAM Toolbox，不启动 AMCL；
- `slam:=false`：加载静态地图并启动 AMCL，不启动 SLAM Toolbox。

官方模拟器 `include` 被包装在 `scoped group` 中；子作用域内分别将 `localization`、`slam` 与 `nav2` 置为 `false`。这一设计既阻止父级 `slam:=true` 泄漏到官方 `spawn` 流程而产生重复节点，也阻止子级参数反向覆盖项目自身的模式选择。

4.3 关键导航参数

表 3 自定义 A* 与 Nav2 关键参数

参数	设置值	作用
<code>planner_plugins</code>	<code>[GridBased]</code>	规划器映射名称
<code>GridBased.plugin</code>	<code>tb4_astar_planner/AStarPlanner</code>	自定义插件类型
<code>allow_unknown</code>	<code>false</code>	不穿越未知栅格
<code>use_diagonal</code>	<code>true</code>	使用八连通搜索
<code>cost_penalty</code>	<code>2.0</code>	膨胀代价惩罚
<code>lethal_cost</code>	<code>253</code>	致命障碍阈值
<code>robot_radius</code>	<code>0.175 m</code>	代价地图机器人半径
<code>inflation_radius</code>	<code>0.45 m</code>	障碍膨胀半径
<code>xy_goal_tolerance</code>	<code>0.25 m</code>	目标位置容差
<code>smoothing_frequency</code>	<code>20 Hz</code>	速度平滑频率

5 实验步骤

5.1 编译与静态验证

```
source /opt/ros/humble/setup.bash
cd experiment2/ros2_ws
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install
source install/setup.bash
```

```
colcon test
colcon test-result --verbose
```

工作空间构建成功, A* 核心算法单元测试通过。运行时进一步检查插件参数、Planner Server lifecycle 状态和 /plan 发布者, 确认路径确由自定义插件生成。

5.2 SLAM 增量建图

启动建图模式:

```
ros2 launch tb4_experiment_bringup simulation.launch.py slam:=true
```

启动后确认仅存在一个 /slam_toolbox, 不存在 /amcl, 且 /map 的唯一发布者为 SLAM Toolbox。机器人通过 /cmd_vel_nav 执行“直行—旋转—直行”的 L 形运动, 速度经 smoother 输出到 /cmd_vel。记录运动前后地图摘要并保存:

```
ros2 run nav2_map_server map_saver_cli \
  -t /map -f experiment2/maps/lab_map
```

5.3 AMCL 初始化与规划器检查

结束建图流程并完全关闭旧 Gazebo、RViz 与 ROS 2 launch 进程, 重新启动导航模式:

```
ros2 launch tb4_experiment_bringup simulation.launch.py slam:=false
```

在 RViz 中点击 2D Pose Estimate, 设定机器人初始位置和朝向, 等待粒子云、Laser-Scan 与静态地图对齐。随后检查:

```
ros2 param get /planner_server planner_plugins
ros2 param get /planner_server GridBased.plugin
ros2 lifecycle get /planner_server
ros2 topic info /plan -v
```

实测返回的插件为 tb4_astar_planner/AStarPlanner, Planner Server 状态为 active [3], /plan 唯一发布者为 planner_server。

5.4 发送导航目标

在 RViz 中使用 Nav2 Goal 将目标设置为约 $(-0.149, -3.924)$ m, 并持续记录 /plan 与 /navigate_to_pose/_action/status。验收条件包括:

1. 路径不少于 3 个位姿，首末点分别靠近机器人和目标；
2. 对 $y \in [-2.3, -1.7]$ 的路径点，均位于 $x \in [-1.9, 1.7]$ 的 shelf 占用带之外；
3. 最大横向坐标绝对值不小于 2.0 m；
4. 同一 UUID 经历 EXECUTING (2) 并进入 SUCCEEDED (4)，不得进入 ABORTED (6)；
5. 最终机器人至目标距离不大于 0.35 m。

6 实验结果与分析

6.1 SLAM 增量建图结果

图 2 与图 3 分别为运动前和完成 L 形运动后的 RViz 状态。初始局部地图已显示机器人附近货架边界；运动后新激光观测扩大了已知区域，机器人模型、LaserScan 与地图保持一致。

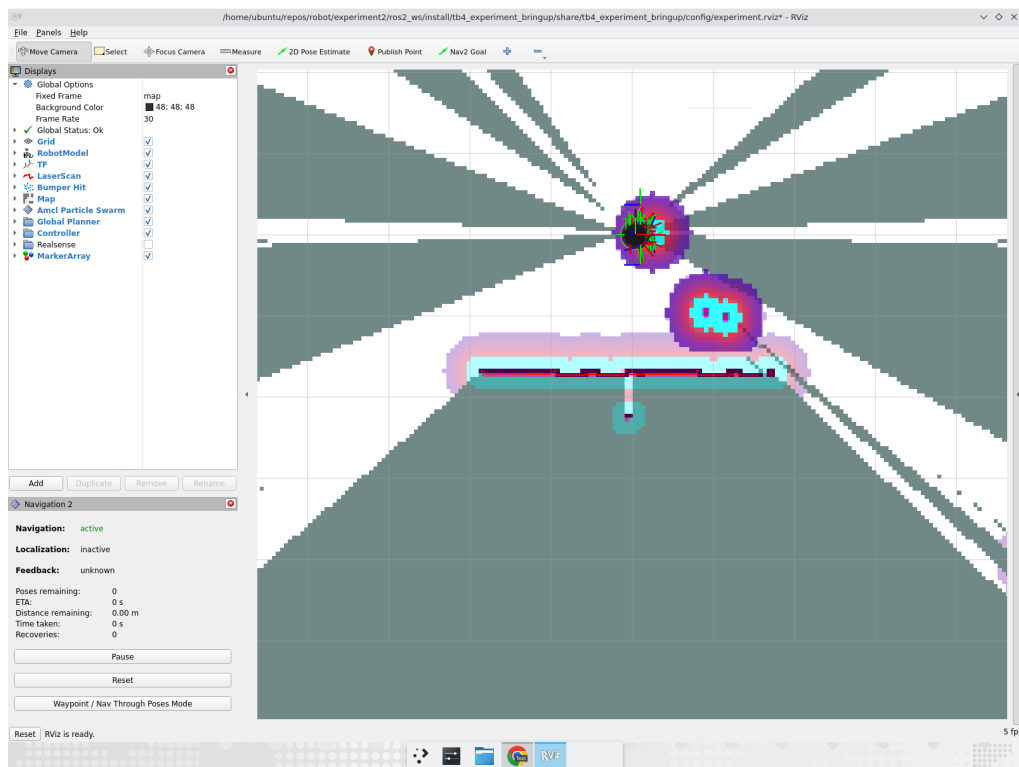


图 2 SLAM 建图运动前的局部地图

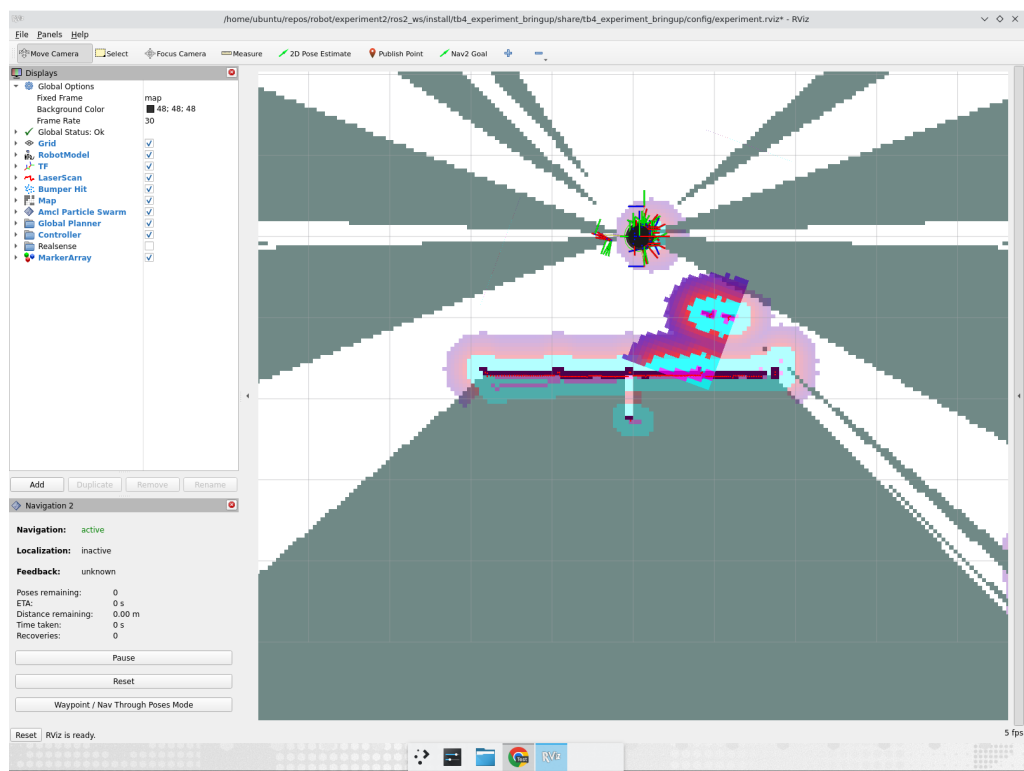


图 3 SLAM 建图运动后的增量地图

增量建图客观指标

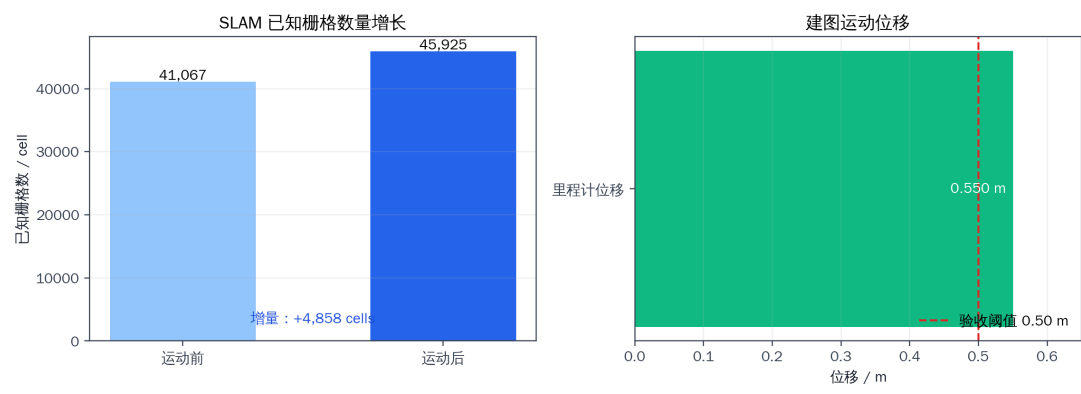


图 4 增量建图客观指标

表 4 给出 SLAM 结果。机器人位移 0.550 m，高于 0.50 m 验收阈值；已知栅格增加 4,858 个，证明地图随观测产生了有效增量。保存地图分辨率为 0.05 m/cell，YAML 正确引用 lab_map.pgm。

表 4 SLAM 流程量化结果

指标	实测值	判定
SLAM Toolbox 节点数	1	通过
AMCL 节点数	0	通过
初始已知栅格数	41,067	—
最终已知栅格数	45,925	—
已知栅格增量	4,858	通过
里程计位移	0.550 m	通过 (阈值 0.50 m)
地图分辨率	0.05 m/cell	通过
地图文件	YAML + PGM	通过
/clock 监听	240.4 s 墙钟, 状态 ok	通过

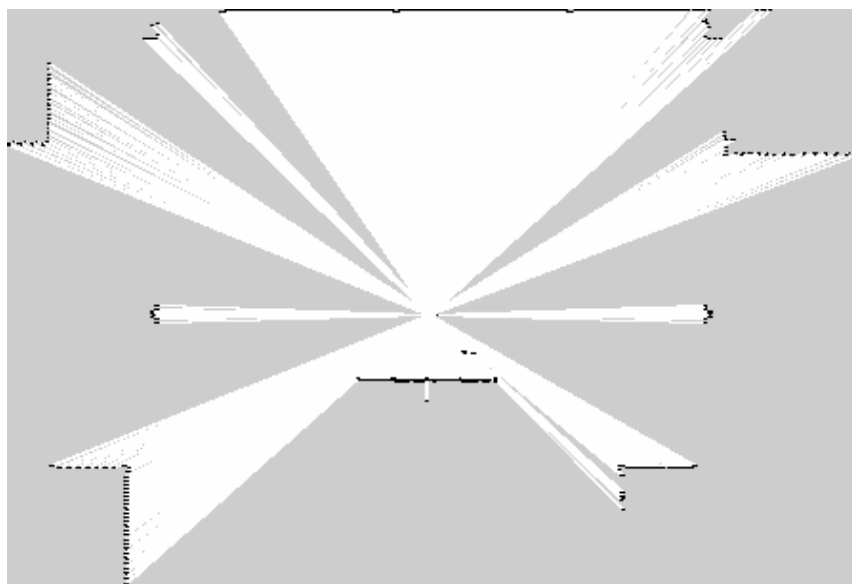


图 5 SLAM 流程保存的占据栅格地图

6.2 AMCL 自定位结果

图 6 中 Navigation 与 Localization 均处于 active 状态, 粒子云集中于机器人附近, 激光点与 warehouse 静态地图边缘对齐。采集的 `/amcl_pose` 消息 frame 为 `map`, 且 `map -> odom -> base_link` TF 链有效, 满足全局规划的坐标基础。

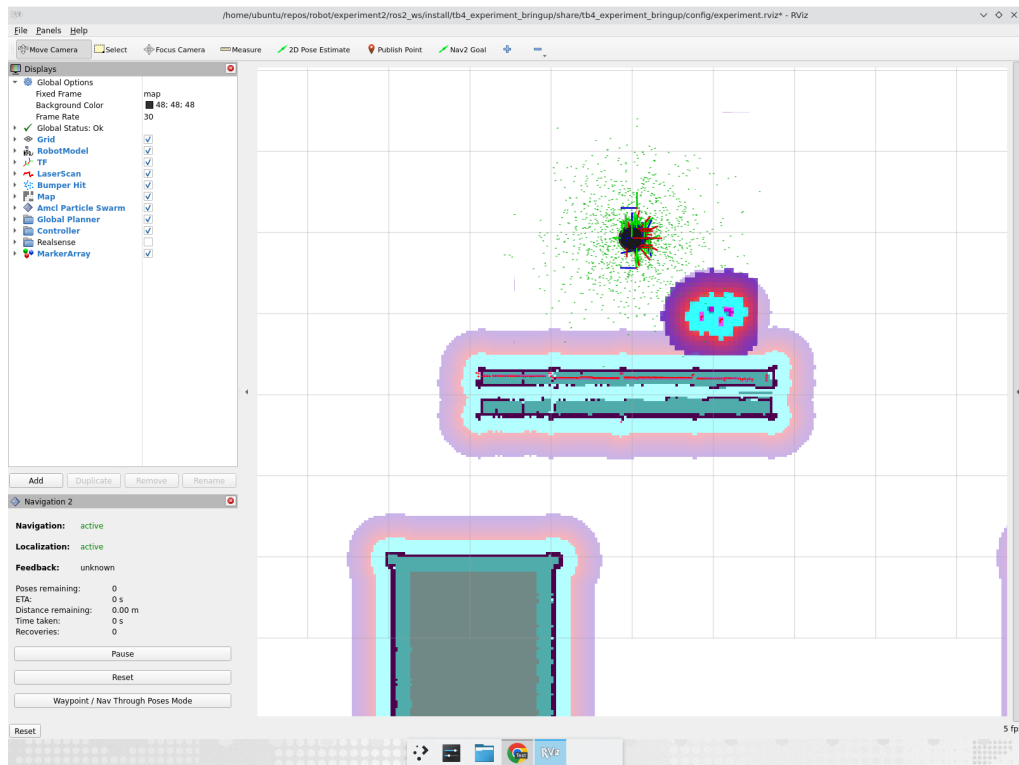


图 6 通过 RViz 设定初始位姿后的 AMCL 收敛状态

6.3 A* 路径与绕障结果

图 7 使用 ROS 2 /plan 原始位姿绘制。路径从机器人附近出发，先向负 x 方向绕到货架端部，再进入目标所在下方走廊；在 $y \in [-2.3, -1.7]$ 范围内，路径点均位于 shelf_7 判定带之外。

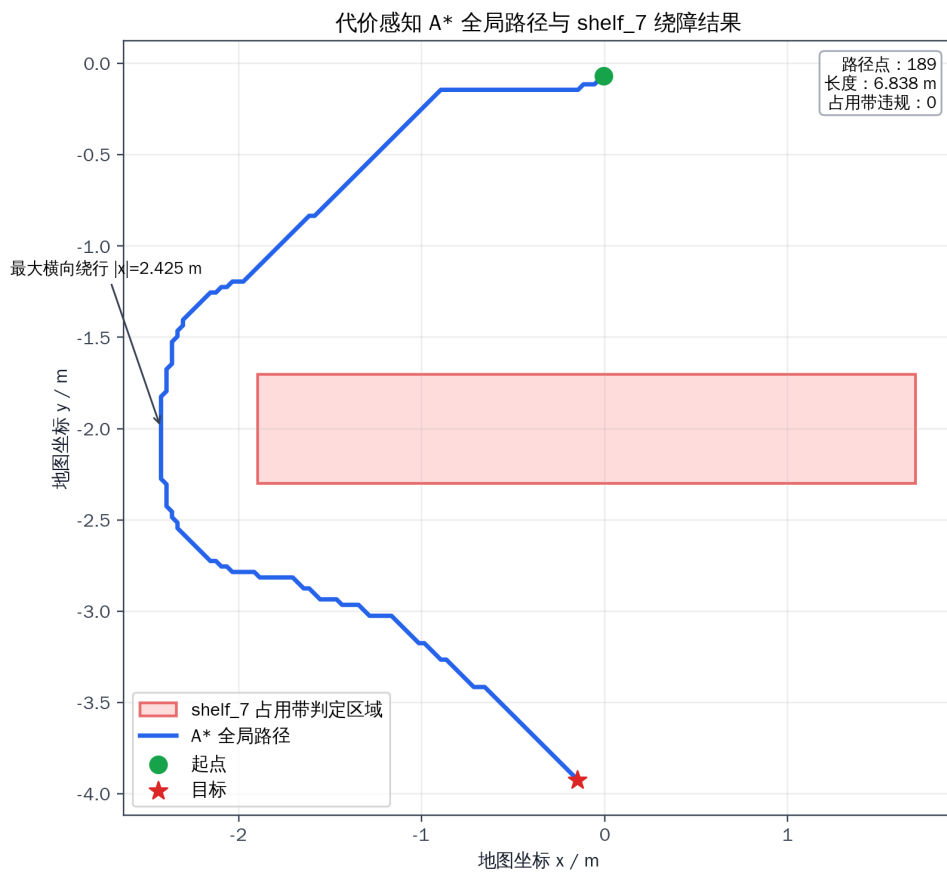


图 7 A* 全局路径与 shelf_7 绕障判定

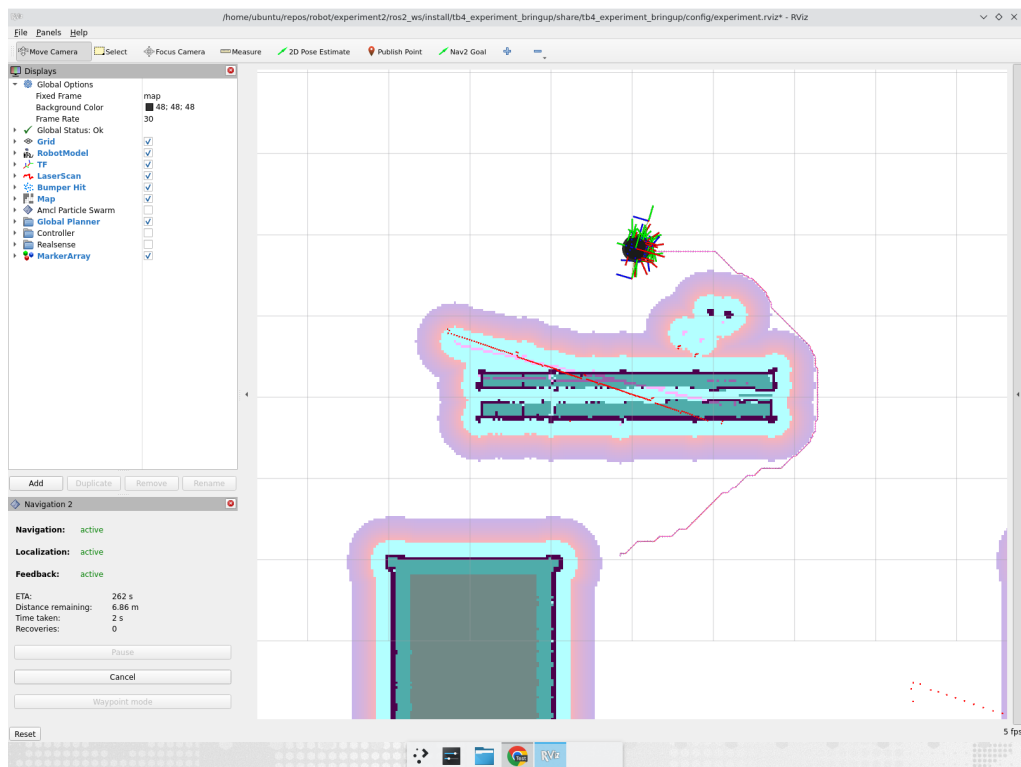


图 8 RViz 中的 A* 路径与导航执行状态

表 5 全局路径量化结果

指标	实测值	判定
路径位姿数量	189	通过
起点	$(-0.005, -0.068)$ m	靠近机器人
终点	$(-0.149, -3.924)$ m	与目标一致
路径长度	6.838 m	—
最大横向绕行	2.425 m	通过 (阈值 2.0 m)
shelf 占用带违规点	0	通过
/plan 发布者	planner_server 唯一发布	通过

6.4 Nav2 导航终态

目标 UUID 缩写为 45d5...0d3e, 完整值保存在提交包的 action 数据文件中。监听到的状态序列为 EXECUTING (2) $\rightarrow$ SUCCEEDED (4), 相同 UUID 未出现 ABORTED (6)。最终机器人位置为 $(-0.204, -3.716)$ m, 目标位置为 $(-0.149, -3.924)$ m; 按平面欧氏距离计算, 二者相距 0.215 m, 小于 0.35 m 验收阈值, 也小于 Nav2 配置的 0.25 m 位置容差。

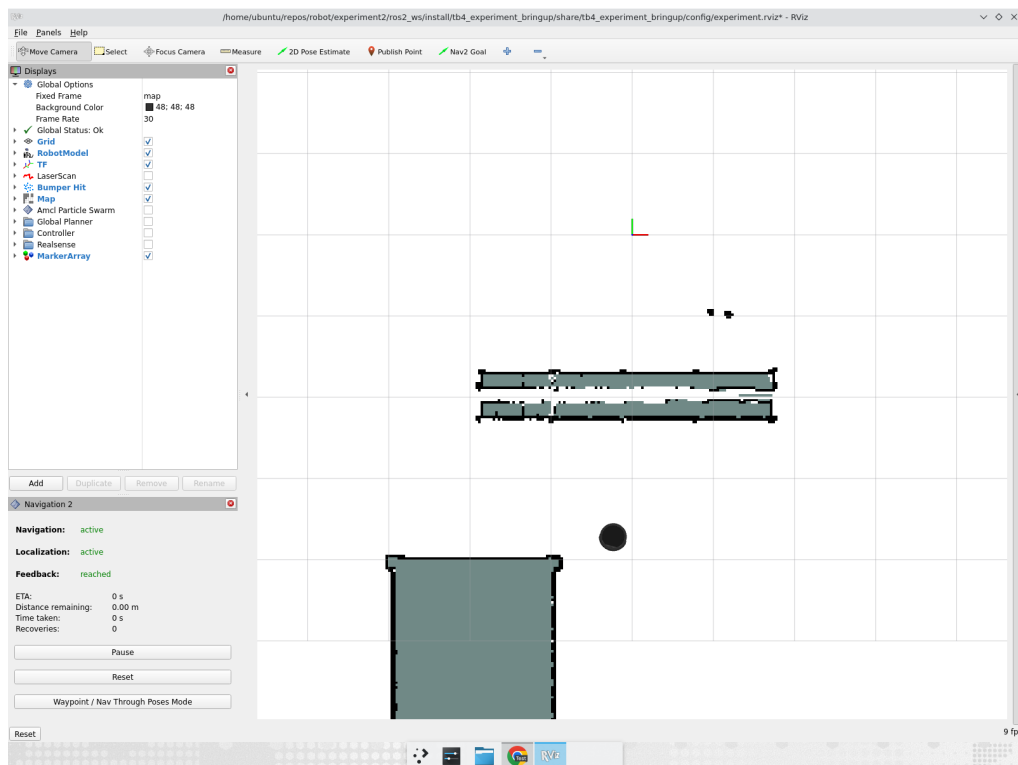


图 9 Nav2 到达目标: Feedback 为 reached, 剩余距离为 0.00 m

表 6 仿真自主导航最终验收

验收项	实测结果	判定
Ignition Gazebo 中机器人实际运动	物理仿真、传感器与速度闭环运行	通过
AMCL 自定位	map frame, TF 有效	通过
自定义全局规划器加载	tb4_astar_planner/AStarPlanner	通过
货架绕障	0 个 shelf 带违规点	通过
Nav2 动作终态	同一 UUID: 2 $\rightarrow$ 4	通过
最终目标距离	0.215 m	通过
仿真时钟严格连续性	出现一次 > 10 s 监听间隙, 随后恢复	性能项未通过

6.5 仿真时钟与低负载配置分析

SLAM 流程的时钟监视器在 240.4 s 墙钟内接收 427 个样本, 仿真时间从 32.5 s 增至 75.1 s, 状态为 ok。导航流程在 303.2 s 墙钟内接收 627 个样本, 监视器最后记录到 74.1 s, 并因连续超过 10 s 未收到回调而标记为 stalled; 之后独立采样得到 103.2 s, 证明 Gazebo 时钟恢复而非永久停止, 机器人也继续运动直至 action 成功。

10 s 是监控判定阈值, 不是 Gazebo 的运行上限, 也不会主动造成时钟停顿。本机仅约 2 vCPU 且无硬件 OpenGL, Gazebo 传感器计算与 RViz costmap、粒子云渲染竞争 CPU, 导致实时率显著低于 1。为获得可完成的端到端实验, 证据采集使用了以下临时低负载配置:

- Gazebo 以 server-only 模式运行, 关闭三维客户端但保留物理、传感器和时钟;
- OAK-D 仿真更新关闭, 因为本实验的 SLAM/Nav2 使用 RPLIDAR;
- RPLIDAR 更新率由默认高频降至 10 Hz, /scan 始终保持发布;
- 关闭 Gazebo 射线绘制, 以及完成相应证据后关闭部分 RViz 视觉叠层;
- 临时 world 的 max_step_size 设为 0.1 s, 以降低物理计算量。

上述措施没有关闭 SLAM、AMCL、Planner Server、Controller Server、Velocity Smoother 或 Nav2 action, 因此不改变自主导航的功能性证据; 但 max_step_size 和激光频率改变了仿真保真度, 本报告不将其表述为默认高保真参数下的实时性能测试。建议在具有硬件 3D 加速、至少 4 核 CPU 和 16 GiB 内存的 Ubuntu 22.04 主机上使用默认 world 与传感器参数复测, 可同时打开 Gazebo GUI 进行三维演示。

7 问题诊断与改进

7.1 启动参数作用域

初始启动文件未显式转发 `gz_args`，导致自定义低负载 world 未被 Gazebo 使用；修复后又发现父级 `slam:=true` 会泄漏至官方子启动，从而产生两套 SLAM 节点。将官方 simulator include 放入 scoped group 后，官方 spawn 的内部参数与项目级 SLAM/AMCL 模式完全隔离。该修改使“一个地图源、一个定位源、一个 Planner Server”成为可重复的启动不变量。

7.2 RViz 视角确定性

官方配置保存了旋转后的 TopDownOrtho 角度，导致屏幕坐标与地图坐标难以对应。实验专用 `experiment.rviz` 固定 Fixed Frame=map、Angle=0、X=0、Y=-1.965，并预置 SetInitialPose 和 GoalTool。每次启动均恢复相同俯视方向，从而使初始位姿和目标点操作可复现。

7.3 后续优化方向

1. 以 lifecycle 状态或进程事件替代固定 5 s 的启动延时，提高慢速主机稳定性；
2. 为 A* 增加起点可通行性、起点等于终点且位于障碍等边界测试；
3. 在高性能主机上恢复默认物理步长、完整 OAK-D 和默认激光频率，测量规划耗时、实时率与资源占用；
4. 使用 SLAM Toolbox 的序列化 pose graph 和 continuation mapping 验证“加载已有地图后继续扩图”的扩展流程；
5. 在实体 TurtleBot4 上配置 DDS/Discovery Server，复用同一 planner 插件和 Nav2 参数进行实机验证。

8 结论

本实验完成了 TurtleBot4 仿真自主导航的完整功能链：Ignition Gazebo 提供机器人、warehouse 环境、激光雷达、里程计与物理运动；SLAM Toolbox 建立并保存增量地图；AMCL 在静态地图上完成自定位；自定义代价感知 A* 以 Nav2 pluginlib 插件形式生成无碰撞全局路径；DWB、速度平滑器和 Create 3 仿真底盘构成闭环执行系统。

实验的量化结果表明, SLAM 已知区域增加 4,858 个栅格, 地图分辨率为 0.05 m/cell; A* 路径包含 189 个位姿、长度 6.838 m, 并以 2.425 m 的横向绕行避开 `shelf_7`; Nav2 同一目标 UUID 从 `EXECUTING` 进入 `SUCCEEDED`, 机器人最终距目标 0.215 m。因此, 课程提出的“在 Ignition Gazebo 仿真环境中实现 TurtleBot4 自主导航”已经完成。

测试主机性能不足造成一次超过 10 s 的 `/clock` 监听间隙, 但时钟随后恢复且导航成功结束。该问题说明系统在低配、软件渲染环境下的实时性仍需优化, 不否定自主导航功能的完成结论。报告对这一限制进行了保守披露, 并提供默认配置复测建议。

参考文献

- [1] Clearpath Robotics, *TurtleBot 4 User Manual: Features*, <https://turtlebot.github.io/turtlebot4-user-manual/overview/features.html>.
- [2] Clearpath Robotics, *TurtleBot 4 User Manual: Networking*, <https://turtlebot.github.io/turtlebot4-user-manual/setup/networking.html>.
- [3] Clearpath Robotics, *TurtleBot 4 User Manual: Discovery Server*, https://turtlebot.github.io/turtlebot4-user-manual/setup/discovery_server.html.
- [4] Open Navigation LLC, *Writing a New Planner Plugin*, https://docs.nav2.org/plugin_tutorials/docs/writing_new_nav2planner_plugin.html.
- [5] S. Macenski and I. Jambrecic, “SLAM Toolbox: SLAM for the dynamic world,” *Journal of Open Source Software*, vol. 6, no. 61, 2021.
- [6] P. E. Hart, N. J. Nilsson, and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [7] S. Macenski, F. Martín, R. White, and J. G. Clavero, “The Marathon 2: A Navigation System,” *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2020.

A 仿真复现命令

A.1 导航模式

```
source /opt/ros/humble/setup.bash
source experiment2/ros2_ws/install/setup.bash
ros2 launch tb4_experiment_bringup simulation.launch.py slam:=false
```

在 RViz 中依次使用 2D Pose Estimate 和 Nav2 Goal。

A.2 建图模式

```
source /opt/ros/humble/setup.bash
source experiment2/ros2_ws/install/setup.bash
ros2 launch tb4_experiment_bringup simulation.launch.py slam:=true
```

A.3 低负载复测模式

低配主机可自行复制官方 warehouse SDF 并降低传感器更新率,然后通过 `gz_args` 指向该临时文件;临时 overlay 不属于课程源码,提交包不将其作为默认运行前提。

```
export AMENT_PREFIX_PATH=/path/to/temporary/overlay:$AMENT_PREFIX_PATH
ros2 launch tb4_experiment_bringup simulation.launch.py \
  slam:=false \
  gz_args:="/absolute/path/to/warehouse-fast.sdf -r -s -v 2"
```

B 报告生成方法

本报告的 PDF 与 Word 均以 `report.tex` 为内容源:

```
sudo apt install pandoc texlive-xetex texlive-lang-chinese python3-docx
cd experiment2/report
make all
```

`make pdf` 使用 XeLaTeX 编译专业排版 PDF; `make docx` 使用 Pandoc 解析同一 LaTeX 源文件,并应用 `reference.docx` 中的中文字体、标题、页边距、页眉和页码样式,生成可编辑的 Word 文档。